



BrightLearn

DATA · ANALYTICS

2026 EDITION · BEGINNER'S GUIDE

SQL for Analysts

The complete beginner's guide to Structured Query Language — from your first **SELECT** to multi-table joins, aggregation, window functions, and best practices for real-world data analysis.

TOPIC

SQL for Data Analysis

ENGINES

Postgres · MySQL · SQLite

LEVEL

Beginner-friendly

INCLUDES

**SELECT · JOIN · GROUP BY ·
CTEs**

BRIGHTLEARN · DATA & ANALYTICS
GUIDE

**Query · Join · Aggregate ·
Decide**

The language data speaks.

00

What is SQL?

A 50-year-old language that quietly runs the modern world — every banking transaction, every Spotify play count, every Amazon order — and the most universally useful skill in data analysis.

DEFINITION

SQL — Structured Query Language

SQL (pronounced "sequel" or "S-Q-L") is a **declarative language for asking questions of relational databases**. Instead of writing step-by-step instructions, you describe *what* you want — "give me the top 10 customers by revenue this year" — and the database engine figures out *how* to fetch it. It is the universal language of structured data.

A BRIEF HISTORY

From IBM lab to every tech stack on Earth

SQL was invented at **IBM in the early 1970s** by Donald Chamberlin and Raymond Boyce, based on the relational model proposed by Edgar F. Codd. Standardised by ANSI in 1986, it has outlived hundreds of programming-language fads. Today it powers Postgres, MySQL, SQLite, SQL Server, Oracle, BigQuery, Snowflake, Redshift — and almost every analytics tool on the market understands it natively.

★ WHY EVERY ANALYST MUST KNOW SQL

Spreadsheets break at a few hundred thousand rows. **SQL handles billions without blinking**. Almost every dashboard, BI tool (Tableau, Power BI, Looker, Metabase), and modern Python or R workflow ultimately runs SQL under the hood. Learning it once unlocks every data tool you'll ever touch.

01 Where SQL is Used

SQL isn't a niche skill for engineers — it's a daily tool for analysts, scientists, marketers, finance teams, and product managers. Here's where it shows up.

◆ ROLES THAT USE SQL EVERY DAY

6 roles

A Data Analyst the daily driver

Pulling reports, building dashboards, answering ad-hoc questions from the business. SQL is the analyst's most-used tool — usually more than Excel and Python combined.

Reports

Dashboards

S Data Scientist model training data

Almost every ML project starts with a SQL query that pulls a training dataset out of a warehouse. Strong SQL = faster, cleaner experimentation cycles.

Feature eng

Sampling

E Data Engineer pipelines & warehouses

Builds and maintains the database systems analysts query. Writes SQL inside ETL pipelines (dbt, Airflow) and designs the schemas everyone else depends on.

ETL

dbt

Schemas

M Marketing & Growth campaign attribution

Funnel analysis, cohort retention, channel attribution — all sit on top of SQL queries against event tables. The marketers who learn SQL stop waiting on the data team.

Funnels

Cohorts

F Finance & Operations revenue & cost reporting

Revenue recognition, cost allocation, reconciliation — SQL replaces fragile spreadsheets that couldn't handle the volume or audit trail required by modern finance teams.

Reconciliation

Audit

P Product Managers feature impact

"Did the new checkout flow actually help conversion?" PMs who can write a SQL query answer this in minutes instead of waiting days for an analyst's bandwidth.

A/B tests

Metrics

★ THE UNIVERSAL SKILL

Of all data skills, SQL is the **most transferable across industries and roles**. The query that pulls customer lifetime value at a fintech is structurally identical to the one pulling student attendance at a university. **Learn it once, use it everywhere.**

02 SQL Database Engines

SQL is a standard, but every database engine has its own quirks and superpowers. Here are the seven you're most likely to meet in the wild.

THE ENGINES THAT RUN THE DATA WORLD

7 engines

P PostgreSQL postgresql.org

The most advanced open-source relational database. Strong standards compliance, rich data types (JSON, arrays, geospatial via PostGIS), and the engine of choice for serious modern analytics.

Open-source

Advanced

Best default

M MySQL mysql.com

The most widely deployed open-source database in the world — the "M" in LAMP stack. Powers WordPress, Shopify, and a huge share of web applications. Owned by Oracle since 2010.

Web

Popular

Mature

S SQLite sqlite.org

A self-contained, serverless database in a single file — the most deployed database on Earth (every iPhone has hundreds). Perfect for learning SQL and for embedded apps.

Single file

Embedded

Learn here first

M Microsoft SQL Server microsoft.com/sql-server

The enterprise standard at most large corporations using the Microsoft stack. Tight integration with Excel, Power BI, and Azure. SSMS is its iconic admin tool.

Enterprise

T-SQL

Power BI

O Oracle Database oracle.com

The grandfather of commercial databases — deeply entrenched in banks, telcos, and government systems. Powerful and expensive. PL/SQL is its procedural extension.

Enterprise

PL/SQL

Banking

B Google BigQuery cloud.google.com/bigquery

A serverless cloud data warehouse that runs SQL queries against petabytes in seconds. Pay-per-query pricing. Hugely popular for marketing analytics and as a Google Analytics 4 destination.

Cloud

Petabyte-scale

Serverless

Snowflake snowflake.com

The cloud data warehouse that defined the modern analytics stack. Separates storage from compute — meaning you only pay for what you query. Used by most modern data teams alongside dbt and Looker.

Cloud warehouse

Modern stack

dbt-native

03 The 5 Sublanguages of SQL

SQL isn't really one language — it's five overlapping ones, each with a distinct job. As an analyst, you'll live in DQL. Engineers and admins handle the rest.

Q DQL — Data Query where analysts live

The **SELECT** statement and everything that supports it — **FROM**, **WHERE**, **JOIN**, **GROUP BY**, **ORDER BY**. This is where 90% of an analyst's work happens.

SELECT **Read-only**

M DML — Data Manipulation change the data

Statements that modify data inside existing tables: **INSERT** adds new rows, **UPDATE** changes existing ones, and **DELETE** removes them. Use with care.

INSERT **UPDATE** **DELETE**

D DDL — Data Definition structure the database

Defines and changes the structure itself: **CREATE TABLE**, **ALTER TABLE**, **DROP TABLE**. Engineers and admins use this; analysts mostly read it.

CREATE **ALTER** **DROP**

C DCL — Data Control permissions & access

Controls who can do what — **GRANT** to give a user permissions, **REVOKE** to take them away. Almost always managed by a DBA.

GRANT **REVOKE** **DBA**

T TCL — Transaction Control commit or roll back

Manages groups of statements that must succeed or fail together. **BEGIN** starts a transaction, **COMMIT** makes the changes permanent, and **ROLLBACK** undoes them if anything went wrong. Critical when modifying data — especially in finance, where a half-completed transfer is worse than no transfer at all.

BEGIN **COMMIT** **ROLLBACK**

★ WHERE TO FOCUS FIRST

As a beginner analyst, spend **95% of your study time on DQL (SELECT and friends)**. Touch DML lightly so you understand how data gets in. You can safely ignore DDL, DCL, and TCL until your role explicitly requires them.

04 SELECT — The Anatomy of a Query

Every analytical query in SQL is essentially a variation on a single sentence. Learn its six clauses in this order, and you've learned 80% of SQL.

```
-- The six clauses, in the order you write them
SELECT  column_a, column_b, COUNT(*) AS total      -- 1. what columns to return
FROM    orders                                    -- 2. which table(s) to read
WHERE   order_date >= '2025-01-01'                -- 3. filter rows BEFORE grouping
GROUP BY column_a, column_b                       -- 4. collapse rows into groups
HAVING  COUNT(*) > 10                             -- 5. filter the GROUPS
ORDER BY total DESC                               -- 6. sort the result
LIMIT   100;                                       -- 7. cap the rows returned
```

★ THE ORDER ENGINES ACTUALLY EXECUTE

You write the clauses in the order above — but the database engine **executes them in a different order**:

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT. That's why a column alias defined in **SELECT** can be used in **ORDER BY** but not in **WHERE**.

04 The Six Clauses Explained

Each clause has a precise job. Master what each one does — and the order they run in — and you can read or write any analytical SQL query.

1 **SELECT — choose your columns**

List the columns to return. Use `*` for all (avoid in production), or apply aggregate functions. Rename outputs with `AS`.

2 **FROM — pick your table(s)**

The source. Joins (`INNER JOIN` , `LEFT JOIN`) attach here to pull data from multiple related tables.

3 **WHERE — filter rows**

Keep only rows matching a condition. Runs *before* grouping — cannot use aggregate functions here.

4 **GROUP BY — collapse rows**

Combine rows sharing the same values into one row per group. Pair with aggregates to answer "per category" questions.

5 **HAVING — filter the groups**

Like `WHERE` , but applied *after* grouping — so it CAN reference aggregates.

6 **ORDER BY & LIMIT — sort and cap**

`ORDER BY ... DESC` sorts the result. `LIMIT 100` caps the rows returned — essential for large tables.

05 Filtering, Sorting & Distinct Values

The day-to-day operators that turn a million-row table into the exact 50 rows you want to see. Memorise these — you'll use them in nearly every query.

◆ WHERE CLAUSE OPERATORS

8 operators

Operator	Example	What it does
<code>=</code> <code>!=</code>	<code>status = 'paid'</code>	Exact match (or not match)
<code>></code> <code><</code> <code>>=</code> <code><=</code>	<code>price >= 100</code>	Numeric and date comparisons
<code>BETWEEN ... AND</code>	<code>age BETWEEN 18 AND 65</code>	Inclusive range — works for numbers and dates
<code>IN (...)</code>	<code>country IN ('UK', 'US', 'DE')</code>	Match any value in a list
<code>LIKE</code>	<code>email LIKE '%@gmail.com'</code>	Pattern match. <code>%</code> = any chars, <code>_</code> = single char
<code>IS NULL</code> <code>IS NOT NULL</code>	<code>phone IS NULL</code>	Test for missing values — never use <code>= NULL</code>
<code>AND</code> <code>OR</code> <code>NOT</code>	<code>paid AND amount > 100</code>	Combine multiple conditions
<code>EXISTS</code>	<code>EXISTS (SELECT 1 FROM ...)</code>	True if a subquery returns any rows

◆ SORTING, DE-DUPLICATING & LIMITING

```
-- Top 10 most recent orders, no duplicate customers
SELECT DISTINCT customer_id, order_date, total
FROM orders
WHERE status = 'paid'
ORDER BY order_date DESC, total DESC
LIMIT 10;
```

★ THREE COMMON BEGINNER TRAPS

- ▶ **NULLs are not zero or empty.** `WHERE phone = NULL` always returns nothing — use `IS NULL`.
- ▶ **String literals use single quotes**, not double. Double quotes are reserved for column names in most engines.
- ▶ **Keywords are case-insensitive**, but string values usually *are* case-sensitive.

06

JOINS — Combining Tables

Real data lives in many tables — customers in one, orders in another, products in a third. JOINS are how you stitch them back together. The four types every analyst must know.

I**INNER JOIN**

only matching rows

Returns only the rows where the key exists in **both** tables. The most common join — used when you only care about records that have data in both sides.

```
SELECT o.id, c.name
FROM   orders o
INNER JOIN customers c
      ON o.customer_id = c.id;
```

L**LEFT JOIN**

all left + matching right

Returns **all rows from the left table**, plus matching rows from the right. Where there's no match, right-side columns come back as **NULL**. Great for finding gaps.

```
SELECT c.name, o.id
FROM   customers c
LEFT JOIN orders o
      ON c.id = o.customer_id;
```

R**RIGHT JOIN**

all right + matching left

The mirror image of LEFT JOIN. Returns all rows from the right table. Rarely used in practice — most analysts just rewrite as a LEFT JOIN with the tables swapped, which reads more naturally.

```
SELECT o.id, c.name
FROM   orders o
RIGHT JOIN customers c
      ON o.customer_id = c.id;
```

F**FULL OUTER JOIN**

everything from both

Returns all rows from **both** tables. Matched rows are combined; unmatched rows on either side fill the missing columns with **NULL**. Useful for reconciliation.

```
SELECT c.name, o.id
FROM   customers c
FULL OUTER JOIN orders o
      ON c.id = o.customer_id;
```

★ THE 30-SECOND MENTAL MODEL

Picture two overlapping circles. **INNER JOIN** = the overlap only. **LEFT JOIN** = the left circle plus the overlap. **RIGHT JOIN** = the right circle plus the overlap. **FULL OUTER JOIN** = both circles entirely. Once you can picture the diagrams, joins stop being scary.

07

Aggregation & GROUP BY

This is where SQL transforms from "find rows" to "answer business questions." Aggregate functions collapse many rows into single summary values per group.

THE FIVE AGGREGATE FUNCTIONS YOU'LL USE DAILY

5 functions

Function	Example	Returns
COUNT(*)	COUNT(*)	Number of rows in the group
COUNT(col)	COUNT(email)	Number of <i>non-NULL</i> values in email
SUM(col)	SUM(revenue)	Total of all numeric values
AVG(col)	AVG(rating)	Arithmetic mean
MIN / MAX(col)	MAX(order_date)	Smallest / largest value — works for numbers, strings, dates

A REAL ANALYST QUERY

```
-- Top 5 product categories by revenue this year, only those
-- with more than 100 orders
SELECT  p.category,
        COUNT(*)           AS total_orders,
        SUM(o.amount)      AS total_revenue,
        ROUND(AVG(o.amount), 2) AS avg_order_value
FROM    orders o
JOIN    products p ON o.product_id = p.id
WHERE   o.order_date >= '2025-01-01'
AND     o.status = 'paid'
GROUP BY p.category
HAVING  COUNT(*) > 100
ORDER BY total_revenue DESC
LIMIT  5;
```

★ THE GOLDEN RULE OF GROUP BY

Every column in your **SELECT** must either appear in the **GROUP BY** clause or be wrapped in an aggregate function. **Forget this once and you'll spend an hour debugging it.** The error messages are famously unhelpful — "column must appear in GROUP BY" usually means you forgot to add it (or wrap it in **SUM**, **MAX**, etc.).

08 Subqueries, CTEs & Window Functions

The three intermediate tools that separate working analysts from beginners. They make complex questions readable — and unlock answers that simple queries can't reach.

◆ SUBQUERIES — QUERIES INSIDE QUERIES

```
-- Find customers who spent more than the overall average
SELECT customer_id, SUM(amount) AS total_spent
FROM orders
GROUP BY customer_id
HAVING SUM(amount) > (SELECT AVG(amount) FROM orders);
```

◆ CTEs — COMMON TABLE EXPRESSIONS (THE READABLE WAY)

```
-- Same answer, but readable: define named steps with WITH
WITH customer_totals AS (
  SELECT customer_id, SUM(amount) AS total_spent
  FROM orders
  GROUP BY customer_id
)
SELECT customer_id, total_spent
FROM customer_totals
WHERE total_spent > (SELECT AVG(amount) FROM orders);
```

◆ WINDOW FUNCTIONS — AGGREGATE WITHOUT COLLAPSING

```
-- Rank each product within its category by revenue
SELECT product_name, category, revenue,
       RANK() OVER (PARTITION BY category ORDER BY revenue DESC) AS rank_in_cat
FROM product_sales;
```

★ WHEN TO USE EACH

- ▶ **Subquery** — small, single-use computation embedded in a larger query.
- ▶ **CTE** — when logic has multiple steps. Reads top-to-bottom like a recipe.
- ▶ **Window function** — running totals, rankings, moving averages — without collapsing rows.

09 Modifying Data, Indexes & Views

Beyond querying, SQL has tools to write data, speed up reads, and save reusable queries. Use them sparingly — with great power comes great risk.

◆ INSERT · UPDATE · DELETE

```
-- 1. INSERT — add a new row
INSERT INTO customers (name, email, country)
VALUES ('Alice Mokoena', 'alice@example.com', 'ZA');

-- 2. UPDATE — change existing rows (always use WHERE!)
UPDATE customers
SET    country = 'South Africa'
WHERE  country = 'ZA';

-- 3. DELETE — remove rows (also: always use WHERE!)
DELETE FROM customers
WHERE  last_login < '2020-01-01';
```

⚠ THE FORGOTTEN WHERE

The single most expensive mistake in SQL is running an **UPDATE** or **DELETE** without a **WHERE** clause — which modifies every row in the table. Always run a **SELECT** with the same **WHERE** first to check what you'll affect. Wrap destructive changes in a transaction so you can **ROLLBACK** if needed.

I Indexes — speed up reads CREATE INDEX

An index is a separate sorted structure on a column — like the index at the back of a book. Queries that filter or join on indexed columns can be 100× to 10,000× faster. The trade-off: writes get slower and storage grows.

B-tree

Faster reads

V Views — saved queries CREATE VIEW

A **VIEW** is a stored **SELECT** that you can query like a table. Use them to encapsulate complex logic (e.g., "active customers") so other queries stay clean and consistent.

Reusable

DRY

M Materialised Views — pre-computed results CREATE MATERIALIZED VIEW

A materialised view stores the *result* of a query physically — so subsequent reads are instant. You refresh it on a schedule. Perfect for expensive dashboards that don't need real-time data. Available in Postgres, Oracle, BigQuery, Snowflake, and most modern engines.

Pre-computed

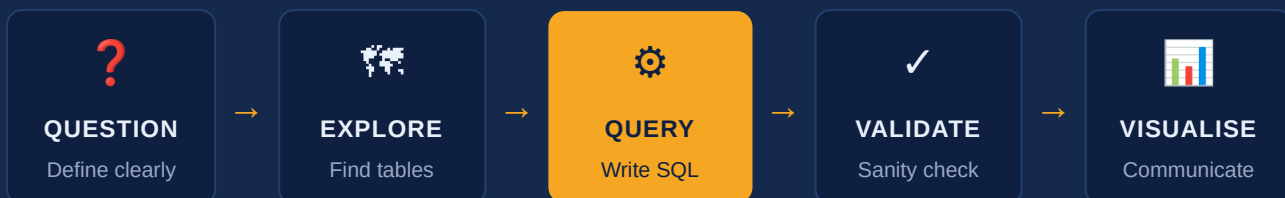
Dashboards

Refresh

10 The SQL Data Analysis Workflow

Every real-world SQL analysis follows the same five steps. Master this loop and you'll feel comfortable in any database, any tool, any company.

FROM QUESTION TO INSIGHT IN 5 STEPS



Iterate freely — validating often reveals the question wasn't quite right and sends you back to step 1.

01 Define the question precisely

"How are sales doing?" is not answerable. "What was total paid revenue per region for January 2026, compared to January 2025?" is. **The hardest part of analysis is asking the right question.**

02 Explore the schema

Find the relevant tables and understand the relationships. Look at sample rows with `SELECT * FROM table LIMIT 10`. Read column names, check data types, identify primary and foreign keys.

03 Write the query — iteratively

Build it up gradually: start with a basic `SELECT` + `LIMIT 10`, add filters, add joins one at a time, add aggregations last. **Run it after every change.**

04 Validate the result

Does the row count make sense? Do the totals match a known number from another source? Spot-check a few rows manually. *If the answer surprises you, the query is probably wrong — not the world.*

05 Visualise & communicate

Push results to Tableau, Power BI, Looker, Metabase, or directly into a Pandas DataFrame for charting. Numbers in tables don't change minds — charts and clear narratives do.

11 Best Practices & Your SQL Stack

The habits that separate junior analysts from senior ones — plus a recommended starter stack for your first real SQL project.

★ THE 8 HABITS OF EFFECTIVE SQL ANALYSTS

- ▶ **Format your queries.** Uppercase keywords, one clause per line, indent joins. Future-you will thank present-you.
- ▶ **Comment the why, not the what.** The SQL explains *what* you did. Comments should explain *why* — the business decision behind the filter.
- ▶ **Always preview before you write.** Run a `SELECT` with the same `WHERE` before any `UPDATE` or `DELETE`.
- ▶ **Use CTEs for anything non-trivial.** A query with three CTEs reads ten times better than a query with three nested subqueries.
- ▶ **Always include a LIMIT during exploration.** A missing `LIMIT` on a 10-billion-row table can lock up the warehouse for everyone.
- ▶ **Use explicit JOINS, never commas.** `FROM a JOIN b ON ...` is clear; `FROM a, b WHERE ...` is a 1990s footgun.
- ▶ **Save your queries.** Use a Git repository, dbt project, or your BI tool's library — not random Slack messages and untitled.sql tabs.
- ▶ **Validate with a second query.** Compute the same number a different way. If they match, you're probably right.

◆ RECOMMENDED BEGINNER SQL STACK



Database

SQLite for learning (zero install)
· PostgreSQL for real projects.



Editor

DBeaver (free, all engines) ·
DataGrip (paid, exceptional).



Visualisation

Metabase for self-serve
dashboards · Tableau / Power
BI for enterprise.

★ COST TO START LEARNING

Everything you need to become job-ready in SQL is **free**: SQLite ships with macOS and Linux, PostgreSQL is open-source, DBeaver is free, and dozens of free practice databases (Northwind, Sakila, the Chinook music store) are available online. **R0. Just time and curiosity.**



Ask better questions.

SQL is not really about syntax — it's about *thinking* in tables, joins, and groups. The syntax takes a week. The thinking takes a career. The fastest way to build it is to pick a real dataset, write one query a day, and force yourself to validate every answer. Do that for ninety days and you'll be more capable than 90% of analysts in your industry.

★ BRIGHTLEARN · SQL FOR ANALYSTS 2026 ★